

Evaluación Integradora 1 - INFB6074: Fundamentos de Arquitectura Computacional, Big Data y Ambientes Reproducibles

Autores: Ignacio Ramírez, Cristian Vergara, Francisco Provoste

Profesor: Dr. Ing. Michael Miranda Sandoval

27 de mayo de 2026

Resumen

Este informe documenta la Evaluación Integradora 1 del curso INFB6074. La Parte I corresponde a seis ejercicios aplicados que cubren los contenidos de las semanas 1 a 5: un bootloader de dos etapas en ensamblador x86 ejecutado en QEMU (Ejercicio 1); un microbenchmark experimental de CPU, memoria, disco y operaciones de entrada y salida (Ejercicio 2); una comparación de procesamiento secuencial y paralelo local sobre 500.000 registros (Ejercicio 3); un pipeline de datos reproducible con arquitectura medallón Bronze-Silver-Gold, calidad, metadatos y linaje sobre dominio astronómico (Ejercicio 4); una mini infraestructura local con Docker Compose, MinIO como data lake y consultas *lakehouse* con DuckDB (Ejercicio 5); y una evaluación comparativa multicriterio de cinco alternativas arquitectónicas Big Data, alimentada con la evidencia experimental de los ejercicios anteriores (Ejercicio 6). La Parte II corresponde al anteproyecto integrador, cuya formulación se presenta de forma resumida en este documento y de forma extensa en su documento independiente. Todos los resultados son reproducibles desde el repositorio acompañante: el bootloader arranca en QEMU mostrando el emblema institucional; el benchmark muestra firmas claras de cada recurso (CPU sostiene 5 millones de pares/s, la jerarquía de memoria se manifiesta como 4-8 veces de razón entre acceso secuencial y aleatorio, y el disco binario supera por dos órdenes de magnitud al CSV); el paralelismo alcanza speedup de 20× con 4 procesos; el pipeline descarta 10,71 % de filas mediante 13 reglas de calidad documentadas; la mini infraestructura levanta dos servicios con red interna y volumen persistente; y el ranking arquitectónico ubica al lakehouse y a la nube en las primeras posiciones para escenarios con crecimiento de volumen y gobernanza.

Índice

1	Resumen Ejecutivo	3
2	Contexto Conceptual	4
3	Ejercicio 1: Mini sistema operativo booteable en ensamblador y QEMU	5
3.1	Objetivo	5
3.2	Fundamento técnico	5
3.3	Decisión arquitectónica: arranque en dos etapas	5
3.4	Implementación de la pantalla institucional	6
3.5	Evidencia de ejecución	6
3.6	Por qué esto no es un sistema operativo	7
4	Ejercicio 2: Diagnóstico experimental de CPU, memoria, disco e I/O	8
4.1	Objetivo	8
4.2	Metodología	8
4.3	Resultados del escenario CPU	8
4.4	Resultados del escenario memoria	9
4.5	Resultados del escenario disco	9
4.6	Conclusión técnica	10
5	Ejercicio 3: Paralelismo local y medición de speedup	11
5.1	Objetivo	11
5.2	Diseño del experimento	11
5.3	Resultados	11
5.4	Discusión: overhead, particionamiento y límites	12
5.5	Conclusión técnica	12
6	Ejercicio 4: Pipeline Big Data Local con Python, Docker, Calidad y Parquet	13
6.1	Objetivo	13
6.2	Dominio y fuentes de datos	13
6.3	Arquitectura medallón y reglas de calidad	13
6.4	Reproducibilidad con Docker	14
6.5	Resultados de ejecución	14
6.6	Distinción entre script y pipeline administrado	15
7	Ejercicio 5: Mini infraestructura con Docker Compose	16
7.1	Objetivo	16
7.2	Decisión arquitectónica: data lake local con MinIO	16
7.3	Composición de servicios	16
7.4	Patrón lakehouse: DuckDB consultando MinIO	17
7.5	Resultados de ejecución	17
7.6	Qué resuelve Docker Compose y qué no	18
8	Ejercicio 6: Evaluación comparativa de arquitecturas Big Data	19
8.1	Objetivo	19
8.2	Alternativas consideradas	19
8.3	Criterios y ponderaciones	19
8.4	Evidencia experimental que alimenta la matriz	20
8.5	Ranking multicriterio	20

8.6	Recomendaciones por escenario	21
8.7	Trade-offs y limitaciones	21
9	Integración y Análisis Conjunto	22
10	Anteproyecto Integrador (resumen)	23
11	Entorno Técnico y Registro Mínimo	24
12	Conclusiones	25
12.1	Aprendizajes integrados	25
12.2	Limitaciones del trabajo	25
12.3	Próximos pasos	25
13	Referencias y Anexos	26
13.1	Estructura del repositorio	26
13.2	Declaración de uso de herramientas de apoyo	26
13.3	Referencias	27

1. Resumen Ejecutivo

La Evaluación Integradora 1 articula los contenidos de las semanas 1 a 5 del curso INFB6074 en seis ejercicios aplicados (Parte I) más un anteproyecto integrador (Parte II). Los seis ejercicios cubren, en una progresión lógica, los niveles del stack que sustentan cualquier infraestructura para ciencia de datos: arranque del sistema (Ejercicio 1), caracterización experimental de los recursos del equipo (Ejercicio 2), paralelismo local con medición de speedup (Ejercicio 3), pipeline reproducible con calidad y linaje sobre dominio astronómico simulado (Ejercicio 4), mini infraestructura local con Docker Compose y patrón *lakehouse* (Ejercicio 5), y evaluación comparativa multicriterio de arquitecturas Big Data alimentada por los hallazgos cuantitativos de los anteriores (Ejercicio 6).

Los principales resultados experimentales son los siguientes. El bootloader de dos etapas funciona en QEMU mostrando el escudo UTEM rasterizado, las identificaciones del curso y del grupo, y cuatro líneas explicativas sobre arquitectura. El benchmark muestra que el procesador sostiene aproximadamente 5 millones de pares por segundo en cargas N-body de orden cuadrático, que el acceso secuencial a memoria es 5,3 veces más rápido que el aleatorio, y que el formato binario supera al CSV por dos órdenes de magnitud en operaciones de disco. El paralelismo con cuatro procesos sobre 500.000 registros reduce el tiempo de 90,82 a 4,46 segundos. El pipeline aplica 13 reglas de calidad sobre dos fuentes y descarta el 10,71 % de las filas por incumplimiento, produciendo tres tablas analíticas en formato Parquet. La mini infraestructura levanta MinIO y un servicio Python comunicados por red interna, con un volumen persistente, y consulta los Parquet directamente desde el data lake con DuckDB. La evaluación arquitectónica ubica en las primeras posiciones del ranking a las arquitecturas cloud y data lake (4,25 y 3,89 respectivamente), y al pipeline Docker columnar implementado en el Ejercicio 4 en tercer lugar con 3,76 puntos.

La arquitectura recomendada depende del escenario: para un equipo pequeño con recursos limitados, un pipeline Docker columnar es la respuesta más equilibrada; para crecimiento de volumen, cloud o lakehouse; para entornos regulados, cloud con gobernanza o pipeline Docker con controles formales; para datos semiestructurados continuos, mini servicios o cloud. El anteproyecto integrador propone una infraestructura de datos personal para priorización contextual de señales heterogéneas, en línea con la arquitectura lakehouse local pero adaptada a un caso de estudio único y con énfasis explícito en soberanía de datos.

2. Contexto Conceptual

La Evaluación Integradora articula los contenidos de las primeras cinco semanas del curso en un único cuerpo de trabajo. Las semanas 1 y 2 introdujeron los fundamentos digitales y la arquitectura de sistemas computacionales (procesadores, jerarquía de memoria, entrada y salida); la semana 3 desarrolló el procesamiento paralelo local; la semana 4 introdujo las arquitecturas Big Data y los modos batch, streaming e híbrido; la semana 5 cerró con administración de datos, calidad, metadatos, linaje, observabilidad y Docker como infraestructura reproducible.

Los seis ejercicios fueron diseñados para ejercitar estos contenidos con criterio experimental, no demostrativo. El Ejercicio 1 toca el nivel más bajo del stack (firmware, sector de arranque, modo real x86), el Ejercicio 2 caracteriza empíricamente los recursos del equipo donde se ejecutará todo lo demás, el Ejercicio 3 introduce la primera capa de paralelismo, los Ejercicios 4 y 5 construyen sucesivamente un pipeline administrado y una infraestructura local, y el Ejercicio 6 cierra integrando la evidencia obtenida en una decisión arquitectónica razonada. La conexión entre ejercicios no es accidental: el dominio astronómico de los Ejercicios 4 y 5 es el mismo (un observatorio simulado tipo Cerro Paranal), el Silver del Ejercicio 4 es la entrada del Ejercicio 5, y la matriz de evaluación del Ejercicio 6 se nutre de los resultados cuantitativos de los Ejercicios 2, 4 y 5.

Este informe documenta cada ejercicio con su objetivo, fundamento técnico, comandos relevantes, evidencia de ejecución, resultados, limitaciones y conclusión. En cada caso se hace explícito qué concepto de infraestructura se está evaluando y qué aprendizaje se extrae. El entorno común a todos los ejercicios es un MacBook con procesador Apple M4, 10 núcleos físicos, 16 GB de RAM, macOS 26.3.1, Python 3.12.13 y Docker Desktop 4.73.0.

3. Ejercicio 1: Mini sistema operativo booteable en ensamblador y QEMU

3.1. Objetivo

Implementar un programa booteable mínimo en ensamblador x86 real mode que se ejecute en QEMU y muestre una pantalla institucional en modo caracteres más elaborada que un simple “Hola mundo”. El ejercicio conecta fundamentos digitales, ciclo de arranque, arquitectura básica, abstracción hardware/software y rol inicial del sistema operativo.

3.2. Fundamento técnico

Al encender la máquina, la CPU salta al *reset vector* (0xFFFFFFF0), donde reside el firmware. El BIOS ejecuta el POST (Power-On Self Test), inicializa dispositivos y construye la tabla de vectores de interrupción. Luego lee el primer sector de 512 bytes del dispositivo de arranque; si los bytes en los offsets 0x1FE y 0x1FF son 0x55 y 0xAA, el sector se considera arrancable, se copia a la dirección física 0x7C00 y se transfiere el control a ese punto. La CPU arranca en **modo real**: registros de 16 bits, 1 MB direccionable, sin protección de memoria, con segmentación ($\text{direccion_fisica} = \text{segmento} \times 16 + \text{offset}$). En este modo es directamente accesible el **framebuffer de texto VGA** en la dirección física 0xB8000, modo de 80 columnas $\times$ 25 filas con 2 bytes por celda (carácter CP437 + atributo de color).

3.3. Decisión arquitectónica: arranque en dos etapas

El enunciado exige una pantalla institucional con emblema UTEM y al menos tres líneas explicativas. Una implementación monolítica supera ampliamente los 510 bytes útiles del sector de arranque; NASM rechazó la primera versión con `TIMES value -163 is negative`. En lugar de recortar el contenido, se adoptó el patrón estándar de la industria: **arranque en dos etapas**, como GRUB o el cargador de Windows.

- **stage1.asm (512 bytes)**: sector de arranque cargado por el BIOS en 0x7C00. Su única responsabilidad es leer la etapa 2 desde el disco con `int 0x13` (función 0x02, modo CHS, 4 sectores desde el sector 2) y saltar a ella. Lleva la firma 0xAA55.
- **stage2.asm (2048 bytes)**: ya sin restricción de 512 bytes, escribe la pantalla institucional directamente al framebuffer 0xB8000.

La imagen de disco final es la concatenación `stage1 + stage2` (2.560 bytes). Esta decisión no es una desviación del enunciado sino su consecuencia lógica: el propio enunciado pide explicar por qué este programa no es un sistema operativo real, y el límite de 512 bytes es precisamente la razón por la que ningún SO arranca en un solo sector.

3.4. Implementación de la pantalla institucional

El emblema UTEM se diseñó rasterizando el logo institucional oficial. El logo se recortó (solo el escudo), se redujo a una grilla de 18 columnas $\times$ 19 celdas y cada píxel se clasificó en tres categorías: vacío, azul institucional (0x09) y verde institucional (0x0A). Para aproximar el contorno curvo del blasón se usaron caracteres de **medio bloque** de CP437 (0xDF mitad superior, 0xDC mitad inferior, 0xDB bloque completo), que duplican la resolución vertical efectiva del modo texto. Cada celda del escudo almacena su propio par [carácter][atributo de color], lo que requirió un segundo motor de render que recorre pares carácter/atributo, distinto del motor de cadenas monocolor usado para el texto.

3.5. Evidencia de ejecución

La compilación produce los binarios con tamaño exacto y firma verificable:

```
Tamano stage1.bin : 512 bytes    (correcto: boot sector)
Tamano stage2.bin : 2048 bytes
Tamano disk.img   : 2560 bytes   (512 + 2048)
Firma offset 0x1FE: 55 aa       (firma 0xAA55 presente)
```

Listing 1: Verificación de los artefactos producidos por el Makefile.

La Figura 1 muestra la ejecución en QEMU. Se observa el escudo UTEM rasterizado con su contorno de blasón y el cuadro verde, el texto “UTEM” grande en blanco, la identificación del curso INFB6074, la carrera, el grupo y las cuatro líneas explicativas sobre CPU, memoria, arranque en dos etapas y escritura directa al framebuffer.



Figura 1: Pantalla institucional renderizada por la etapa 2 del bootloader en QEMU.

3.6. Por qué esto no es un sistema operativo

El programa carece de gestión de procesos, planificador, gestión de memoria, sistema de archivos, controladores de dispositivos y modo protegido. Un sistema operativo real, tras la etapa de arranque, cargaría un kernel, activaría el modo protegido o largo, configuraría tablas de paginación y cedería el control a un planificador. Aquí, la etapa 2 dibuja una pantalla y detiene la CPU con `cli + hlt`. Lo que el ejercicio sí demuestra es el contrato fundamental que hace posible que cualquier SO exista: el firmware carga un primer sector de 512 bytes en `0x7C00`, valida la firma `0xAA55` y cede el control; ese primer sector, demasiado pequeño para hacer algo útil por sí mismo, carga código adicional desde el disco. Comprender ese mecanismo es entender el “paso uno” sobre el que se construye toda la pila de software, incluida la infraestructura de ciencia de datos.

4. Ejercicio 2: Diagnóstico experimental de CPU, memoria, disco e I/O

4.1. Objetivo

Construir un microbenchmark local en Python que aísle cuatro recursos del equipo (CPU, memoria con tres patrones de acceso, disco binario y disco CSV) y exhiba la firma característica de cada uno mediante medición experimental, con repetición y cuantificación de variabilidad.

4.2. Metodología

Se diseñaron cuatro escenarios. **CPU N-body**: simulación gravitacional con complejidad $O(N^2)$ sobre $N \in \{1500, 2000, 2800, 4000, 5500\}$ partículas, lo que mide la capacidad aritmética sostenida del procesador. **Memoria**: suma de un arreglo con tres patrones de acceso (secuencial, *strided*, aleatorio) sobre lados $N \in \{500, 1200, 2800, 6000, 11000\}$, lo que expone la jerarquía de caché y el prefetcher. **Disco binario**: escritura y lectura de archivos `.npy` de 2.500 a 8.000 MB para superar el *page cache* y exponer la velocidad real del SSD. **Disco CSV**: misma operación pero con archivos de 200 a 800 MB en formato texto, ya que la serialización CSV es CPU-bound y escala mal con el tamaño. Cada combinación se repitió 8 veces (el enunciado exige al menos 3) con cooldown de 10 segundos entre repeticiones para mitigar el *thermal throttling*. Las semillas aleatorias están fijas.

La función `medir` envuelve cada operación con `time.perf_counter` y registra el delta de memoria RSS con `psutil`. En el escenario de memoria, el arreglo y los vectores de índices se construyen una sola vez por tamaño, de modo que la única variable medida sea el patrón de acceso. Los tres patrones usan la misma primitiva `arr[indices].sum()`, variando solo el orden de los índices, para que la comparación sea justa. Los datos se persisten en `resultados_benchmark.csv` (288 mediciones).

4.3. Resultados del escenario CPU

La Tabla 1 muestra el comportamiento de la simulación N-body.

Cuadro 1: CPU N-body: tiempo y throughput por tamaño (8 repeticiones).

N	Tiempo (s)	σ_t (s)	Throughput (pares/s)	σ_{tp}
1.500	0,253	0,009	4.449.845	152.183
2.000	0,417	0,004	4.790.679	51.660
2.800	0,793	0,004	4.938.659	25.429
4.000	1,599	0,007	5.002.136	21.676
5.500	3,026	0,012	4.997.513	20.305

El throughput se estabiliza alrededor de $5,0 \times 10^6$ pares por segundo a partir de $N = 2,800$. La desviación estándar disminuye con el tamaño, lo que es coherente: las cargas cortas son

más sensibles al ruido del sistema operativo y a transientes del procesador, mientras que las cargas largas promedian esas perturbaciones. La firma del recurso es clara: la curva tiempo-vs- N es estrictamente cuadrática y la pendiente del logaritmo respecto a N se aproxima a 2, como predice la complejidad $O(N^2)$ de la simulación.

4.4. Resultados del escenario memoria

La Tabla 2 compara los tres patrones.

Cuadro 2: Memoria: throughput (MB/s) por patrón de acceso y tamaño.

Lado N	Secuencial	Strided	Aleatorio
500	1.959	1.182	710
1.200	3.725	1.470	1.005
2.800	3.957	2.147	706
6.000	6.108	2.559	1.340
11.000	8.197	2.925	1.549

La firma de la jerarquía de memoria se manifiesta nítidamente. El acceso secuencial alcanza 8.197 MB/s en el tamaño máximo porque el prefetcher de hardware acierta sistemáticamente y la línea de caché se aprovecha por completo. El acceso *strided* (saltos a paso fijo) baja a 2.925 MB/s: el prefetcher acierta menos y aparecen *cache misses* en cada línea no contigua. El acceso aleatorio se queda en 1.549 MB/s, aproximadamente **5,3 veces más lento que el secuencial**, porque cada acceso genera un fallo de caché que obliga a ir a memoria principal. Esta razón de $5\times$ es exactamente la lección teórica de la jerarquía de memoria expresada en datos del equipo: no es la cantidad de bytes leídos lo que importa, sino el patrón con el que se acceden.

4.5. Resultados del escenario disco

La Tabla 3 muestra la diferencia entre formato binario y CSV.

Cuadro 3: Disco: throughput de escritura y lectura (MB/s) por formato y tamaño.

Tamaño	Esc. binario	Lec. binario	Esc. CSV	Lec. CSV
Pequeño (2.500 / 200 MB)	2.182	2.874	12,7	57,1
Medio (4.000 / 400 MB)	2.055	2.156	12,7	56,4
Grande (6.000 / 600 MB)	1.710	1.831	12,6	56,0
Muy grande (8.000 / 800 MB)	1.618	1.081	12,6	55,4

Dos observaciones cruciales. Primero, el throughput binario es **dos órdenes de magnitud superior al CSV** (2.182 MB/s vs. 12,7 MB/s en escritura): la serialización a texto convierte la operación en CPU-bound y el SSD nunca llega a saturarse. Esta es la justificación cuantitativa de por qué los pipelines de datos modernos prefieren formatos binarios como Parquet, ORC o Avro sobre CSV. Segundo, el throughput binario cae al

crecer el archivo: para archivos pequeños (2.500 MB), el sistema operativo aprovecha el *page cache* (16 GB de RAM) y el “disco” parece más rápido de lo que es; para archivos grandes (8.000 MB), el cache no alcanza y se observa la velocidad real del SSD (1.081 MB/s lectura). Esta degradación es la firma del nivel de almacenamiento más cercano al sistema.

4.6. Conclusión técnica

Los cuatro escenarios cumplen su propósito didáctico: muestran que cada recurso tiene una firma medible, que el cuello de botella depende del recurso y no del programa, y que las decisiones de diseño de datos (formato binario vs. texto, patrón de acceso a memoria, escala del trabajo CPU) tienen consecuencias cuantitativas grandes. Los resultados son específicos del equipo y no se deben extrapolar como absolutos, pero las razones relativas ($5\times$ secuencial vs. aleatorio, $150\times$ binario vs. CSV) son lecciones transferibles. Las visualizaciones (`visualizaciones/01-04.png`) ilustran cada firma.

5. Ejercicio 3: Paralelismo local y medición de speedup

5.1. Objetivo

Comparar una solución secuencial con una solución paralela local sobre una tarea de procesamiento de datos repetible, midiendo **speedup** ($S_p = T_1/T_p$) y **eficiencia** ($E_p = S_p/p$) con al menos dos configuraciones de paralelismo, y discutir las razones por las que el speedup rara vez es lineal.

5.2. Diseño del experimento

Se generó un dataset sintético de 500.000 registros con `numpy.random` (semilla 42), con columnas `user_id`, `value`, `category`, `text` y `timestamp`. La tarea repetible es una transformación intensiva en CPU por registro: para cada fila se calcula una suma trigonométrica de 50 términos (`sin` y `cos` con desplazamientos) y un hash SHA-256 del texto concatenado con el identificador. Esta carga se eligió deliberadamente CPU-bound para que el paralelismo muestre impacto real; una carga liviana habría sido dominada por el overhead.

La versión secuencial usa `df.apply` de pandas. La versión paralela usa `joblib.Parallel` con backend `loky`, que crea procesos separados (no hilos), liberándose del Global Interpreter Lock (GIL) de Python. Los registros se convierten a lista de `dict` para que sean serializables entre procesos. Se ejecutaron tres configuraciones: $p = 1$ (secuencial), $p = 2$ y $p = 4$.

5.3. Resultados

La Tabla 4 muestra los tiempos medidos.

Cuadro 4: Tiempos, speedup y eficiencia con `joblib + loky` sobre 500.000 registros.

Procesos p	Tiempo (s)	Speedup S_p	Eficiencia E_p
1	90,82	1,00	1,00
2	8,15	11,14	5,57
4	4,46	20,35	5,09

Los números merecen una explicación. El speedup observado con $p = 2$ es 11,14, lo que matemáticamente excede el ideal lineal (que sería 2). El motivo no es que el paralelismo multiplique mágicamente el trabajo: es que la versión secuencial (`df.apply`) sufre un overhead enorme de pandas en cada fila, mientras que la versión paralela opera sobre una lista plana de `dict` y aprovecha el paso al backend Loky sin esa fricción. La comparación es justa en términos del enunciado (medir secuencial vs. paralelo sobre la misma tarea), pero el speedup “superlineal” refleja también la diferencia entre dos motores de iteración, no solo el efecto puro de los procesos.

Lo que sí es interpretable como efecto puro del paralelismo es el paso de $p = 2$ a $p = 4$: el tiempo cae de 8,15 a 4,46 segundos, una mejora de $1,83\times$ al duplicar el número de

procesos. Esto es coherente con un modelo de Amdahl con fracción serial baja y overhead pequeño en el régimen medido.

5.4. Discusión: overhead, particionamiento y límites

El experimento ilustra cuatro factores que limitan el speedup en la práctica.

Overhead de serialización. Pasar 500.000 `dict` a procesos hijos implica copiar y serializar (*pickle*) los datos. Para tareas livianas, este overhead domina; aquí la tarea es lo suficientemente pesada (operaciones trigonométricas más hash) como para que el costo se diluya, pero está presente.

Granularidad de la tarea. La función se eligió costosa por diseño. Si fuera liviana (por ejemplo, sumar dos números), el overhead de coordinación por registro superaría el cómputo útil y el paralelismo perdería. La regla práctica es: el costo del trabajo paralelizable debe superar el overhead de coordinación por al menos un orden de magnitud.

Memoria compartida vs. copia. Cada proceso recibe copias de los registros: no hay memoria compartida en este diseño. Para datasets que no caben en memoria por triplicado, se usaría `multiprocessing.shared_memory` o `numpy.memmap`, o un framework como Dask que evita la duplicación.

Casos donde paralelizar no mejora. Tres situaciones lo desaconsejan: cargas dominadas por I/O (los hilos serían mejor opción, no los procesos), cargas con sincronización fuerte entre tareas (el costo de coordinación supera el beneficio) y datasets muy pequeños (el overhead de levantar procesos no se amortiza).

5.5. Conclusión técnica

El ejercicio cumple su propósito: muestra que el paralelismo local con procesos puede acelerar significativamente cargas CPU-bound pesadas, pero que el speedup observado depende no solo del número de procesos sino del baseline elegido, del overhead de serialización y de la granularidad de la tarea. La ley de Amdahl establece un techo asintótico determinado por la fracción serial irreductible; en este experimento, el ajuste de Amdahl extendido (con overhead) ofrecería una estimación precisa de ese techo si se midieran más configuraciones de p .

6. Ejercicio 4: Pipeline Big Data Local con Python, Docker, Calidad y Parquet

6.1. Objetivo

Implementar un pipeline reproducible que ingeste dos fuentes heterogéneas, aplique reglas de calidad documentadas, almacene los resultados en formato columnar y entregue tablas analíticas consultables. El pipeline debe ejecutarse en un contenedor Docker con versiones fijas de librerías, incluir catálogo de datos y registro de linaje, y producir un reporte de calidad por ejecución.

6.2. Dominio y fuentes de datos

El dominio aplicado es un observatorio astronómico simulado, calibrado con los rangos reales del Observatorio Cerro Paranal (ESO Chile, 2.635 m sobre el nivel del mar). Las dos fuentes se generaron con `generar_datos.py` siguiendo los rangos físicos del sitio (temperatura nocturna 14-20 °C, humedad 3-20 %, presión 700-900 hPa, *seeing* mediano 0,64 *arcsec*) e inyectando un 10 % de errores controlados para ejercitar las reglas de calidad. La Tabla 5 las resume.

Cuadro 5: Fuentes ingestadas por el pipeline.

Archivo	Formato	Filas	Contenido
lecturas_sensores.csv	CSV	10.125	Mediciones ambientales cada 5 min, 5 sensores, 3 días. Campos de temperatura, humedad, viento, presión, <i>seeing</i> y transparencia.
eventos_observacion.jsonl	JSONL	1.020	Observaciones telescópicas con identificador, instrumento, coordenadas celestes, duración y estado.

6.3. Arquitectura medallón y reglas de calidad

Se adoptó la arquitectura Bronze→Silver→Gold. **Bronze** lee con `read_csv` y `read_ndjson` y persiste en Parquet sin transformación. **Silver** aplica las reglas de calidad, descarta inválidos, tipa columnas y persiste Parquet limpio. **Gold** ejecuta tres consultas DuckDB sobre el Silver y produce tres tablas analíticas. La capa transversal está formada por `metadata/catalogo.json` (esquema declarado por fuente con tipo, obligatoriedad y regla), `metadata/linaje.json` (registro por ejecución de filas leídas, escritas y descartadas) y `metrics/reporte_calidad.json` (conteo por regla).

Se implementaron 13 reglas: 8 sobre lecturas y 5 sobre eventos. La Tabla 6 las resume con su conteo real obtenido en la ejecución del 20 de mayo de 2026.

Cuadro 6: Reglas de calidad aplicadas en el Silver.

Regla	Descripción	Inválidos	%
<i>Lecturas de sensores</i>			
<code>error_humedad_fuera_rango</code>	<code>humedad_pct >100</code> (físicamente imposible)	126	1,24 %
<code>error_sensor_id_nulo</code>	Identificador obligatorio nulo	129	1,27 %
<code>error_timestamp_futuro</code>	Timestamp en año 2027	125	1,23 %
<code>error_seeing_negativo</code>	<code>seeing_arcsec <0</code>	127	1,25 %
<code>error_presion_nivel_mar</code>	<code>presion_hpa >900</code> (descalibrado a 2.635 m)	128	1,26 %
<code>error_transparencia_invalida</code>	<code>transparencia >1.0</code>	128	1,26 %
<code>error_lectura_duplicada</code>	(<code>sensor_id</code> , <code>timestamp</code>) duplicado	260	2,57 %
<i>Eventos de observación</i>			
<code>error_instrumento_invalido</code>	Instrumento fuera del catálogo	21	2,06 %
<code>error_declinacion_invalida</code>	Fuera del rango $[-90^\circ, +90^\circ]$	20	1,96 %
<code>error_timestamp_futuro</code>	Timestamp en año 2027	20	1,96 %
<code>error_sensor_id_nulo</code>	Identificador obligatorio nulo	20	1,96 %
<code>error_obs_id_duplicado</code>	Identificador de observación repetido	40	3,92 %

6.4. Reproducibilidad con Docker

El `Dockerfile` fija `python:3.12-slim` como base y declara las dependencias en `requirements.txt` con versiones exactas (polars 1.40.1, pandas 3.0.3, pyarrow 24.0.0, duckdb 1.5.2). Los comandos de construcción y ejecución son dos:

```
docker build -t pipeline-infb6074 .
docker run --rm -v $(pwd):/work pipeline-infb6074
```

Listing 2: Comandos de construcción y ejecución del pipeline.

El montaje de volumen escribe los resultados en el host, permitiendo inspeccionar las carpetas `bronze/`, `silver/`, `gold/`, `metadata/` y `metrics/` sin entrar al contenedor. Docker aporta tres cosas concretas: aislamiento de dependencias, reproducibilidad de entorno entre sistemas operativos, y comparación controlada (al modificar una regla, solo cambia lo que se quiso cambiar).

6.5. Resultados de ejecución

La Tabla 7 sintetiza los resultados.

Cuadro 7: Resultados del pipeline (corrida del 20/05/2026 17:18:49).

Fuente	Entrada	Válidas (Silver)	Tasa de error
Lecturas de sensores	10.125	9.048	10,64 %
Eventos de observación	1.020	903	11,47 %
Total	11.145	9.951	10,71 %

La zona Gold produjo `condiciones_por_noche.parquet` (35 filas, agrupando por ubicación y noche), `resumen_sensores.parquet` (5 filas, una por sensor) y `observaciones_por_instrumento.parquet` (9 filas). El sensor con mejor *seeing* promedio resultó SENS-004 Oeste con 0,635 *arcsec*; el instrumento más productivo fue el Espectrógrafo con 222 observaciones completadas. La tasa de error global del 10,71 % es coherente con el 10 % de errores inyectados por el generador.

6.6. Distinción entre script y pipeline administrado

Un script que procesa datos y un pipeline administrado se distinguen por la presencia de cuatro elementos: zonas (bronze, silver, gold), reglas declarativas (catálogo + reglas de calidad), metadatos versionados (catálogo y linaje en JSON) y reproducibilidad (Docker + requirements fijos). Este ejercicio implementa los cuatro; un script equivalente con `pandas.read_csv` seguido de transformaciones inline produciría el mismo resultado funcional pero no el mismo resultado operacional: no sería rastreable, no sería reproducible entre máquinas y no se podrían auditar los cambios.

7. Ejercicio 5: Mini infraestructura con Docker Compose

7.1. Objetivo

Diseñar y ejecutar una mini infraestructura local con múltiples servicios usando Docker Compose, integrando un servicio de almacenamiento, un servicio de procesamiento Python, una red interna entre ellos y un volumen persistente, y demostrar el patrón *lakehouse* consultando archivos Parquet sin descargarlos.

7.2. Decisión arquitectónica: data lake local con MinIO

Se eligió MinIO como almacenamiento en lugar de una base relacional. La razón es directa: el output natural del pipeline del Ejercicio 4 son archivos Parquet, no filas relacionales. MinIO es un *object storage* S3-compatible que admite archivos arbitrarios y expone una API idéntica a la de Amazon S3, lo que permite que DuckDB lo consulte con su extensión `httpfs`. La combinación MinIO + DuckDB sobre Parquet es la forma más pequeña posible de materializar el patrón **lakehouse**: almacenamiento de archivos baratos y consulta SQL sobre ellos sin moverlos.

7.3. Composición de servicios

La Tabla 8 describe la infraestructura del `docker-compose.yml`.

Cuadro 8: Componentes de la mini infraestructura.

Componente	Tipo	Puerto	Función
<code>minio</code>	Servicio	9000 / 9001	Data lake S3-compatible (API y consola web).
<code>python_app</code>	Servicio	—	Sube los Parquet al bucket y los consulta in situ.
<code>red_infb6074</code>	Red bridge	—	Resolución por nombre entre servicios.
<code>minio_data</code>	Volumen	—	Persistencia entre ciclos de up/down.

El servicio MinIO usa la imagen oficial `minio/minio` con dos puertos publicados (9000 para la API S3 y 9001 para la consola web), variables de entorno para credenciales (`admin / secreto123`) y el volumen `minio_data` montado en `/data`. El servicio Python se construye desde un `Dockerfile` local con `python:3.12-slim` y las librerías `minio`, `duckdb` y `pandas`. Tiene `depends_on: minio` para que Docker lo levante en orden correcto.

La red bridge `red_infb6074` aísla la comunicación entre los dos contenedores. Dentro de la red, cada servicio es accesible por su nombre: el script Python se conecta a `minio:9000` y Docker resuelve internamente ese nombre. Esta resolución automática de nombres es una ventaja operativa concreta de Compose frente a levantar contenedores manualmente.

7.4. Patrón lakehouse: DuckDB consultando MinIO

El núcleo técnico del ejercicio es la consulta in situ:

```
conn = duckdb.connect()
conn.execute("""
    INSTALL httpfs;
    LOAD httpfs;
    SET s3_endpoint= 'minio:9000';
    SET s3_access_key_id= 'admin';
    SET s3_secret_access_key= 'secreto123';
    SET s3_use_ssl= false;
    SET s3_url_style= 'path';
""")
df = conn.execute("""
    SELECT sensor_id, ubicacion,
    ROUND(AVG(seeing_arcsec), 3) AS seeing_promedio
    FROM read_parquet('s3://sensores-astronomicos/lecturas_limpias.parquet')
    GROUP BY sensor_id, ubicacion
    ORDER BY seeing_promedio ASC
""").df()
```

Listing 3: DuckDB consultando Parquet directamente desde MinIO.

La consulta no descarga el archivo entero: gracias a las estadísticas embebidas en el Parquet, solo se traen los bloques estrictamente necesarios. En entornos productivos, este patrón se reproduce sustituyendo MinIO por S3, GCS o Azure Blob, y DuckDB por Spark, Trino o Athena.

7.5. Resultados de ejecución

La Tabla 9 muestra las cifras del data lake.

Cuadro 9: Resultados observados en el data lake MinIO.

Métrica	Valor
Lecturas almacenadas en el bucket	9.048
Eventos almacenados en el bucket	903
Total de registros en el data lake	9.951
Mejor sensor (<i>seeing</i>)	SENS-004 Oeste, 0,635 <i>arcsec</i>
Instrumento más productivo	Espectrógrafo, 222 observaciones completadas

La consola web en <http://localhost:9001> permite verificar visualmente el bucket `sensores-astronomicos` y los dos objetos Parquet. Al ejecutar `docker compose down` y luego `up` de nuevo, los datos persisten en el volumen.

7.6. Qué resuelve Docker Compose y qué no

Lo que resuelve. Reproducibilidad de entorno entre máquinas, comunicación entre servicios sin configuración manual de red, persistencia local desacoplada de los contenedores, arranque ordenado por dependencias (`depends_on`), y descripción completa de la infraestructura en un único archivo versionable.

Lo que no resuelve. Compose no es una plataforma de gobernanza (no provee control de acceso por rol, no audita cambios al catálogo, no aplica retención ni cifrado en tránsito por defecto). No resuelve seguridad productiva (las credenciales viajan en variables de entorno declaradas en el YAML). No aporta escalabilidad distribuida (corre en un único nodo, no maneja réplicas ni balanceo al estilo de Kubernetes o Spark). No provee monitoreo continuo (si MinIO cae, no hay alarma sin políticas `restart` explícitas). Tratar Compose como sustituto de estas funciones sería un error de criterio.

8. Ejercicio 6: Evaluación comparativa de arquitecturas Big Data

8.1. Objetivo

Construir una evaluación comparativa de alternativas arquitectónicas Big Data a partir de evidencia experimental y criterios de selección, calcular un ranking multicriterio y recomendar arquitecturas para escenarios típicos. Este ejercicio integra los resultados cuantitativos de los ejercicios 2, 4 y 5 con la clasificación de arquitecturas vista en las semanas 4 y 5.

8.2. Alternativas consideradas

Se compararon cinco arquitecturas:

- **A - Monolito archivos:** procesamiento local con scripts Python sobre archivos locales sin contenedores.
- **B - Pipeline Docker columnar:** la arquitectura del Ejercicio 4 (Bronze→Silver→Gold con Parquet y DuckDB en Docker).
- **C - Mini servicios containerizados:** la arquitectura del Ejercicio 5 (Compose + MinIO + Python + red interna).
- **D - Data lake / lakehouse:** extensión conceptual de C a un *object storage* distribuido con motores analíticos remotos.
- **E - Cloud o híbrida:** servicios gestionados en una nube pública (S3 + Glue/EMR/Athena, o equivalentes).

8.3. Criterios y ponderaciones

Se definieron diez criterios con escala de 1 (peor) a 5 (mejor), ponderados según la siguiente prioridad: rendimiento (0,16), escalabilidad (0,15), costo relativo (0,12), gobernanza (0,12), mantenibilidad (0,10), reproducibilidad (0,10), seguridad (0,10), resiliencia (0,08), complejidad operativa (0,05) y madurez requerida del equipo (0,02). Las ponderaciones priorizan rendimiento, escalabilidad, costo y gobernanza porque son las dimensiones más determinantes al seleccionar infraestructura Big Data en contextos reales. Seguridad y resiliencia mantienen peso relevante; complejidad y madurez se ponderan bajo porque son síntomas, no causas.

8.4. Evidencia experimental que alimenta la matriz

La matriz no se construyó con puntuaciones a ojo: cada celda se justifica con evidencia. La Tabla 10 resume los hallazgos cuantitativos utilizados.

Cuadro 10: Evidencia experimental que alimenta la matriz del Ejercicio 6.

Fuente	Hallazgo	Valor
Ejercicio 2	Throughput CPU estable	4.860.892 pares/s
Ejercicio 2	Razón binario / CSV en escritura	151,5×
Ejercicio 2	Razón secuencial / aleatorio en memoria	5,3×
Ejercicio 3	Speedup paralelo con 4 procesos	20,35×
Ejercicio 4	Filas válidas en Silver (lecturas + eventos)	9.951
Ejercicio 4	Tasa de error promedio detectada	10,71 %
Ejercicio 4	Tablas analíticas Gold generadas	3
Ejercicio 5	Total de registros en data lake MinIO	9.951

8.5. Ranking multicriterio

La Tabla 11 muestra el ranking obtenido al aplicar las ponderaciones sobre la matriz.

Cuadro 11: Ranking multicriterio de las cinco alternativas evaluadas.

Arquitectura	Score	Soporte
E - Cloud / híbrida	4,25	Conceptual; extensión natural de B y C.
D - Data lake / lakehouse	3,89	Ejercicios 4 y 5 aportan metadatos, linaje, calidad y patrón lakehouse local.
B - Pipeline Docker columnar	3,76	Ejercicios 2, 4 y 5 muestran la ventaja del formato columnar y la reproducibilidad con Docker.
C - Mini servicios containerizados	3,54	Ejercicio 5 muestra orquestación con Compose y patrón lakehouse minimalista.
A - Monolito archivos	2,16	Bajo en reproducibilidad, gobernanza y escalabilidad; útil solo para prototipado inicial.

8.6. Recomendaciones por escenario

Equipo pequeño con recursos limitados. Se recomienda **B (Pipeline Docker columnar)** cuando el equipo puede asumir la curva de aprendizaje de Docker; **A (Monolito)** solo si el alcance es exploratorio y los resultados no necesitan ser reproducibles fuera de la máquina del autor. La inversión en Docker se amortiza rápidamente.

Organización con crecimiento de volumen. Se recomienda **E (Cloud / híbrida)** si se acepta el costo recurrente y se valora el escalado elástico, o **D (Data lake / lakehouse)** si se prefiere autohospedaje. Ambas evitan los cuellos de botella de un nodo único y aceptan crecimiento horizontal.

Organización regulada. Se recomienda **E con foco en gobernanza**, aprovechando los servicios gestionados de catálogo, IAM y auditoría que ofrecen las nubes públicas, o **B con controles formales** de linaje, catálogo y revisión de cambios al esquema. Priorizar gobernanza aunque suba el costo es la respuesta correcta en este escenario.

Datos semiestructurados o eventos continuos. Se recomienda **C (Mini servicios)** o **E (Cloud)**. Las arquitecturas por servicios facilitan la ingesta continua y el procesamiento por micro-lotes; las nubes ofrecen servicios gestionados específicos para flujos de eventos (Kinesis, Pub/Sub, Event Hubs).

8.7. Trade-offs y limitaciones

Tres limitaciones de la comparación merecen quedar registradas. Primero, las puntuaciones son heurísticas: aunque están ancladas en evidencia experimental, dependen del juicio del evaluador y de las prioridades del contexto. Segundo, las ponderaciones reflejan una visión particular del problema; en entornos regulados, gobernanza y seguridad podrían recibir un peso significativamente mayor, y el ranking cambiaría. Tercero, las arquitecturas conceptuales (D, E) requieren diseño detallado en cualquier implementación real (qué servicios, qué proveedor, qué configuración), de modo que su comparación aquí es de alto nivel.

La recomendación metodológica para una organización que adoptara este marco sería: realizar experimentos controlados para medir rendimiento, costo y resiliencia con cargas representativas del propio dominio, recalibrar la matriz con esos números, y repetir el ranking antes de cualquier decisión de inversión.

9. Integración y Análisis Conjunto

Los seis ejercicios se conectan entre sí en cinco niveles, no como compartimentos estancos.

Nivel 1: Arquitectura computacional como prerrequisito. El Ejercicio 1 (bootloader) ejercita el contrato entre firmware, CPU y memoria que hace posible que cualquier software exista. El Ejercicio 2 caracteriza empíricamente los recursos de la misma máquina donde correrá todo lo demás. Sin entender que el SSD es $150\times$ más rápido en binario que en CSV, no se entiende por qué los pipelines del Ejercicio 4 prefieren Parquet.

Nivel 2: Procesamiento como punto medio. El Ejercicio 3 demuestra que el paralelismo local puede acelerar cargas CPU-bound considerablemente ($20\times$ con 4 procesos) pero también explica por qué no siempre paga: el overhead de coordinación, la granularidad de la tarea y el tipo de carga determinan si paralelizar es la respuesta correcta.

Nivel : Pipeline administrado. El Ejercicio 4 introduce el salto cualitativo de “script que mueve datos” a “pipeline con contrato”. Las zonas, las reglas, el catálogo y el linaje son las cuatro piezas que distinguen una operación profesional de un script descartable.

Nivel 4: Infraestructura local reproducible. El Ejercicio 5 toma la salida del Ejercicio 4 y la ubica en una mini infraestructura containerizada, evidenciando que el patrón *lakehouse* puede materializarse en un equipo único con MinIO + DuckDB + Compose. La transición Silver $\rightarrow$ data lake muestra que la salida administrada de un pipeline puede ser la entrada de una arquitectura distribuida sin transformaciones intermedias.

Nivel 5: Decisión arquitectónica con evidencia. El Ejercicio 6 cierra el ciclo, convierte los hallazgos cuantitativos de los Ejercicios 2, 4 y 5 en puntuaciones de una matriz multicriterio, calcula un ranking y emite recomendaciones por escenario. Es la transición desde el saber-hacer experimental hacia el saber-decidir arquitectónico.

El aporte conjunto del bloque es mostrar que las decisiones de infraestructura no son cuestión de moda tecnológica sino de articulación entre carga, restricciones, evidencia y criterios. Los seis ejercicios no se entienden por separado; se entienden como una progresión de seis pasos hacia la capacidad de fundamentar una arquitectura.

10. Anteproyecto Integrador (resumen)

La Parte II de la evaluación se desarrolla en su documento independiente. Se resume aquí su formulación para preservar la coherencia del informe.

Título tentativo. Infraestructura de datos personal para priorización contextual en flujos de trabajo técnico multi-dominio.

Problema. Un trabajador del conocimiento técnico opera diariamente sobre múltiples sistemas heterogéneos (correo, calendario, mensajería, repositorios, plataformas académicas) sin una capa unificada que normalice, contextualice y priorice las señales generadas. El foco del anteproyecto es la capa de datos, es decir, ingesta, normalización, calidad, almacenamiento mixto, linaje y observabilidad, no la construcción de un asistente conversacional. Un agente de lenguaje aparece únicamente como consumidor que demuestra el valor de la infraestructura.

Arquitectura preliminar. Cinco capas: ingesta multi-fuente con modos batch y near-real-time; almacenamiento mixto con MinIO (Bronze), Parquet (Silver), DuckDB (Gold), vector store (Chroma) y state store (Redis o SQLite); procesamiento con orquestación ligera y workers de calidad y enriquecimiento; consumo mediante una API local (FastAPI); capa transversal de catálogo, linaje, observabilidad y seguridad con principio de *no egress*.

Alternativas consideradas. Tres aproximaciones: motor centralizado (PostgreSQL con extensiones), lakehouse local containerizado (la propuesta) y SaaS comercial de *second brain*. La selección recae en la segunda por restricción de soberanía de datos y por diversidad de cargas.

Validación. El prototipo cubriría ingesta de dos fuentes reales (*mbox* y *ics*), pipeline Bronze→Silver→Gold, las seis reglas de calidad, catálogo en YAML, linaje en JSON, indexación semántica básica en Chroma y una API mínima en FastAPI. Streaming en tiempo real, observabilidad completa y el agente LLM consumidor quedan descritos pero no implementados.

Riesgos identificados. Validez externa (caso de estudio único), fragilidad de conectores ante cambios de APIs externas, privacidad por mezcla de dominios corporativos y personales, riesgo de *feature creep* hacia un asistente personal, y calidad de embeddings limitada por mezcla de idiomas.

El documento completo del anteproyecto se entrega como archivo independiente.

11. Entorno Técnico y Registro Mínimo

La Tabla 12 consigna el entorno común a los seis ejercicios.

Cuadro 12: Entorno experimental común a los seis ejercicios.

Componente	Versión / Detalle
Sistema operativo	macOS 26.3.1 (arm64)
Procesador	Apple M4
Núcleos físicos / lógicos	10 / 10
RAM total	16,0 GB
Almacenamiento total	228,27 GB
Python	3.12.13
NumPy	2.4.5
Pandas	3.0.3
Matplotlib	3.10.9
psutil	7.2.2
polars	1.40.1
pyarrow	24.0.0
duckdb	1.5.2
minio (SDK Python)	7.2.15
joblib	(versión por defecto)
Docker Desktop	4.73.0
Docker Engine	29.4.3
QEMU	qemu-system-x86_64 (Ejercicio 1)
NASM	2.16.01 (Ejercicio 1)

Tamaños de datasets. Ejercicio 2: 288 mediciones repartidas en 4 escenarios. Ejercicio 3: 500.000 registros sintéticos generados con `numpy.random` (semilla 42). Ejercicio 4: 10.125 lecturas (CSV) y 1.020 eventos (JSONL); $8 + 5 = 13$ reglas de calidad. Ejercicio 5: $9.048 + 903 = 9.951$ registros en el data lake. Ejercicio 6: matriz 5 arquitecturas $\times$ 10 criterios.

Reproducibilidad. Cada ejercicio incluye instrucciones de reproducción en su README. Los Ejercicios 4 y 5 se reproducen con un único comando `docker compose up -build` o `docker build + docker run` respectivamente. El Ejercicio 1 se reproduce con `make all`. El Ejercicio 2 se reproduce ejecutando el notebook `actividad2.ipynb` (advertencia: 1 h 20 min en la máquina de referencia debido a los cooldowns). El Ejercicio 3 se reproduce en su notebook `ejercicio-3.ipynb`. El Ejercicio 6 se reproduce ejecutando `ejercicio-6.ipynb` después de los demás (depende de sus archivos de salida).

12. Conclusiones

12.1. Aprendizajes integrados

El trabajo deja cuatro lecciones que atraviesan los seis ejercicios.

Las firmas de los recursos son medibles. El Ejercicio 2 demuestra que CPU, memoria y disco tienen comportamientos cuantitativamente distintos: 5 millones de pares por segundo en CPU sostenida, $5,3\times$ de razón entre acceso secuencial y aleatorio a memoria, $150\times$ entre disco binario y CSV. Estas razones no son curiosidades académicas: explican por qué los pipelines del Ejercicio 4 usan Parquet y no CSV, por qué el Ejercicio 5 puede consultar archivos remotos sin descargarlos.

El paralelismo paga, pero no siempre. El Ejercicio 3 muestra que pasar de 1 a 4 procesos puede multiplicar el rendimiento por 20 en cargas CPU-bound pesadas con backend adecuado, pero también que la elección del baseline, el overhead de serialización y la granularidad de la tarea determinan si el paralelismo conviene. No hay receta universal.

Un pipeline es más que un script. Los Ejercicios 4 y 5 evidencian que la diferencia entre ambos no es la cantidad de líneas de código, sino la presencia de zonas, reglas, catálogo, linaje y reproducibilidad. Estas cuatro piezas convierten una operación de datos en un activo operacional auditable.

Las decisiones arquitectónicas se fundamentan, no se eligen por moda. El Ejercicio 6 cierra el bloque articulando los hallazgos cuantitativos en una matriz multicriterio. La conclusión es que cada arquitectura tiene un escenario donde brilla y otros donde no; el rol del ingeniero es identificar el escenario real y elegir la que lo sirve mejor.

12.2. Limitaciones del trabajo

Tres limitaciones generales acompañan el bloque. Primero, todos los experimentos se ejecutaron en un único equipo (Apple M4, 16 GB RAM, sin GPU CUDA), de modo que los números absolutos no se extrapolan; las razones relativas sí. Segundo, los datasets del Ejercicio 4 son sintéticos calibrados con rangos reales, no datos reales: la dinámica de los errores en datos reales puede ser más sutil. Tercero, las arquitecturas conceptuales del Ejercicio 6 (lakehouse, cloud) no se implementaron, y su comparación con las arquitecturas implementadas se hace en términos de criterios cualitativos más que de mediciones directas.

12.3. Próximos pasos

Los siguientes pasos lógicos sobre este trabajo serían: ejecutar los Ejercicios 4 y 5 sobre un dataset mayor (orden de millones de filas) para observar dónde se rompen las soluciones locales; medir el Ejercicio 3 con configuraciones adicionales de p y ajustar un modelo de Amdahl con overhead, como se hizo en la Actividad 3 del semestre; y desarrollar el prototipo del anteproyecto integrador siguiendo el plan de cinco hitos descrito en su documento.

13. Referencias y Anexos

13.1. Estructura del repositorio

```
EVALUACION-1-INF6074/
+- README.md
+- parte_1_ejercicios/
| +- ejercicio_01_bootloader_qemu/
| | +- Makefile, README.md
| | +- src/ (stage1.asm, stage2.asm)
| | +- build/ (stage1.bin, stage2.bin, disk.img)
| | +- docs/ (informe.md, capturas/qemu-boot.png)
| +- ejercicio_02_benchmark_arquitectura/
| | +- benchmark.py, entorno.py, actividad2.ipynb
| | +- entorno_experimental.md, resultados_benchmark.csv
| | +- visualizaciones/ (01-04.png)
| +- ejercicio_03_parallelismo_local/
| | +- ejercicio-3.ipynb
| +- ejercicio_04_pipeline_bigdata_docker/
| | +- Dockerfile, requirements.txt
| | +- pipeline.py, generar_datos.py
| | +- etapas/ (bronze, silver, gold, calidad, metadata)
| | +- datos/, bronze/, silver/, gold/
| | +- metadata/ (catalogo.json, linaje.json)
| | +- metrics/ (reporte_calidad.json)
| +- ejercicio_05_docker_compose_infraestructura/
| | +- Dockerfile, docker-compose.yml, requirements.txt
| | +- scripts/cargar_datos.py
| | +- datos/ (Silver heredado del Ej.4)
| +- ejercicio_06_evaluacion_arquitectonica/
|   +- ejercicio-6.ipynb
+- parte_2_anteproyecto/
  +- documento_anteproyecto.pdf
  +- slides_anteproyecto.pdf
```

Listing 4: Estructura completa del repositorio.

13.2. Declaración de uso de herramientas de apoyo

Para la organización del informe y la mejora de redacción se utilizó como herramienta de apoyo Claude (Anthropic). El diseño experimental, la implementación del código, la elección de tecnologías, la definición de reglas de calidad, la ejecución de las mediciones, la construcción de la matriz arquitectónica y la formulación del anteproyecto son trabajo del grupo, considerando que se usó IA para asistir en la corrección del mismo.

13.3. Referencias

- Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.
- Patterson, D. A., & Hennessy, J. L. (2021). *Computer Organization and Design RISC-V Edition*. Morgan Kaufmann.
- Stallings, W. (2018). *Computer Organization and Architecture* (11th ed.). Pearson.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- Reis, J., & Housley, M. (2022). *Fundamentals of Data Engineering*. O'Reilly Media.
- Hwang, K., Dongarra, J., & Fox, G. (2013). *Distributed and Cloud Computing*. Morgan Kaufmann.
- DAMA International. (2017). *DAMA-DMBOK: Data Management Body of Knowledge* (2nd ed.). Technics Publications.
- Docker Inc. *Docker Documentation*. <https://docs.docker.com/>
- Docker Inc. *Compose specification*. <https://docs.docker.com/compose/compose-file/>
- QEMU Project. *QEMU Documentation*. <https://www.qemu.org/docs/master/>
- NASM Project. *NASM Documentation*. <https://www.nasm.us/docs.php>
- Python Software Foundation. *Python Documentation*. <https://docs.python.org/3/>
- Pandas Developers. *Pandas Documentation*. <https://pandas.pydata.org/docs/>
- NumPy Developers. *NumPy Documentation*. <https://numpy.org/doc/>
- Polars Developers. *Polars User Guide*. <https://docs.pola.rs/>
- Apache Arrow Developers. *Apache Arrow and Parquet documentation*. <https://arrow.apache.org/docs/>
- DuckDB Foundation. *DuckDB Documentation*. <https://duckdb.org/docs/>
- DuckDB Foundation. *httpfs extension*. <https://duckdb.org/docs/extensions/httpfs.html>
- MinIO Inc. *MinIO Documentation*. <https://min.io/docs/>
- Joblib Developers. *Joblib Documentation*. <https://joblib.readthedocs.io/>
- European Southern Observatory. *Paranal Ambient Conditions Database*. https://archive.eso.org/wdb/wdb/asm/dimm_paranal/form